FIELDOPS

22-week learning to mastery roadmap

JavaScript / TypeScript / Node.js / PostgreSQL
React / React Native / Micro frontends

One shared product. Two different contributions.

Build a field-service platform: dispatchers manage jobs and stock on web; technicians work on mobile; a Node API and PostgreSQL protect business state.

Neha | 5 years

Job workflows, dispatcher UI and mobile completion. GitHub: Neha2212-vi.

Madhavi | 10 years

Inventory workflows, reporting and mobile stock requests. GitHub: madhavichauhan1995.

Both learn every concept. Feature ownership changes the implementation ask, not the mastery requirement. The final capstone crosses feature boundaries.

22 weeks **44 complementary weekly contributions** **338 concepts**

Reviewer: Prem-Tomar. Only the reviewer merges to master.
Pace: 20-25 hours per week per learner; extend a gate when gaps remain.
Status: planning outcomes, not a claim of completed mastery.

Source: updated roadmap review in PR #15, revision b28ca07. Existing week-one issues are assigned to the two learners. Further issue creation remains paused.

Shared learning. Distinct work.

Every week has a job-side contribution and an inventory-side contribution that combine into new product behavior. Reuse accepted units instead of reimplementing the peer's feature.

1. STUDY AND PREDICT

Read the week's concepts. Explain them in plain language and predict examples before running code.

2. BUILD AND INTEGRATE

Implement your own feature, import existing domain/contracts code and demonstrate its integration.

3. REVIEW AND VERIFY

Review the peer's decisions, then independently adapt a bounded behavior or experiment for every assigned concept.

4. PASS AN UNSEEN VARIATION

The coach changes an input, timing or constraint. Each learner diagnoses/adapts independently; assistance is recorded.

Concept index: each week names its newly focused concepts; later work revisits them. The glossary explains all 338 concepts and identifies the first planned focus. Early references preview deeper topics without marking them mastered.

JUMP TO A WEEK

01 JavaScript product foundations

02 JavaScript depth and bounded work

03 TypeScript fundamentals and domain states

04 TypeScript advanced contracts and runtime validation

05 Small Node API and light PostgreSQL

06 React foundations with real reads

07 React forms, effects and failure recovery

08 React server state and optimistic changes

09 Advanced React rendering and measured performance

10 React Native foundations

11 Mobile media, persistence and interaction

12 Node and PostgreSQL correctness: deep focus

13 Durable offline queues and native boundaries

14 Offline conflict resolution and resumable uploads

15 Recoverable live events and background work

16 Node streams, workers and resource limits

17 Web micro frontends and mobile feature modules

18 Micro frontend compatibility and failure recovery

19 Security and end-to-end performance

20 Observability, deployment and database recovery

21 Compatible product evolution and disputed work

22 Independent mastery and product acceptance

Frontend preparation: actual web/mobile implementation issues include screen images and relevant loading, empty, error, success, offline and conflict states. This PDF plans the work; it does not substitute for those designs.

Task/PR policy: function-level inputs, outputs, invalid cases and progressive hints belong in each future task. Tests and documentation are not standalone assignments; test-writing feedback is optional in feature PRs.

PostgreSQL grows with the product

Start with domain contracts, then a small Node API before building clients. Learn SQL against real product reads; commit stock only after the transaction gate.

WEEKS 1-4 | NO DATABASE YET

Use fixture data while mastering JavaScript and TypeScript; agree domain and transport contracts.

Both demonstrate: Both can explain accepted inputs, state transitions and numeric boundaries.

WEEKS 5 | LIGHT FOUNDATION

Tables/rows, primary and foreign keys, simple constraints, INSERT/SELECT/UPDATE/DELETE, parameterized queries, a small repository and initial migration. Start a reproducible local database and a small connection pool. Add basic resource authorization with the first endpoints.

Both demonstrate: Create and read a valid job and part; reject an invalid/unauthorized request. Keep real stock commitment behind the later transaction gate.

WEEKS 6-11 | EVERYDAY PRODUCT QUERIES

Joins, filtering, ordering, aggregates and stable pagination; extend the schema for pending requests and attachment metadata. Use constrained, small migrations. Learn a basic index from an actual list query rather than adding indexes everywhere.

Both demonstrate: Both independently write and explain a feature query and its data constraints; client screens consume its actual result.

WEEKS 12 | HEAVY CORRECTNESS WORK

Atomic multi-step transactions, conditional updates/row locks, isolation, concurrent reservations/completions, deadlocks and bounded retries, idempotency keys/fingerprints and unique constraints. Separate pool wait time from query time.

Both demonstrate: With stock 5, concurrent requests for 3 and 3 accept at most one; stock never becomes negative. Replay of an accepted request has no duplicate effect. Both demonstrate different transaction paths.

Key terms: a constraint rejects invalid data; a transaction commits related changes together; isolation governs concurrent observations. An index speeds selected reads at a write/storage cost.

PostgreSQL grows with the product

Use product failures and actual workloads to justify deeper database work. Device-local persistence and server-side PostgreSQL have different roles.

WEEKS 13-14 | OFFLINE RECONCILIATION

PostgreSQL version checks, optimistic concurrency, audit/conflict records and durable server receipts. Distinguish these from the device's local database transactions and migrations.

Both demonstrate: An offline stale completion/reservation produces an explicit domain conflict; retries preserve accepted work and audit history.

WEEKS 15-18 | RELIABLE DELIVERY AND SCALE

Transactional outbox, consumer deduplication, event cursors, report/database cursors, streaming and bounded buffering; investigate indexes/query plans on the real event/report workloads.

Both demonstrate: Crash after database commit does not lose eventual event delivery; duplicate deliveries do not duplicate effects; export memory stays bounded.

WEEKS 19-20 | HEAVY PERFORMANCE AND RECOVERY WORK

EXPLAIN/EXPLAIN ANALYZE on safe representative workloads, index selectivity/tradeoffs, contention and pool sizing, migration/backfill safety, database privileges, backups, restore verification and recovery objectives.

Both demonstrate: Both measure and improve a different query/transaction, then recover data and verify business invariants after a migration or failure.

WEEKS 21-22 | MASTERY UNDER CHANGE

Backward-compatible schema evolution, compensating operations, auditability and diagnosis of unseen concurrency/performance/recovery failures.

Both demonstrate: Each independently defends a database decision and solves a new product incident; accepted data remains correct.

Core correctness example: stock 5, two requests for 3: accept at most one, never go negative and never apply an accepted request twice. Completion consumes a reservation without deducting the same stock again.

WEEK 01

JavaScript product foundations

NEHA | Job-side contribution

Implement the job-input parser, allocation preview, job view, draft queue and lookup controller from existing issues #1-7.

MADHAVI | Inventory-side contribution

Implement the quantity boundary, pricing, immutable stock reservation, catalog and submit controller from existing issues #8-14.

WHY THIS WEEK / PRODUCT PROGRESS

One composed fixture-driven job and inventory workflow.

CONCEPTS TO UNDERSTAND - BOTH LEARNERS

values and types: A value is data; its type determines which operations make sense. JavaScript has primitive values and objects.

coercion and strict equality: Coercion converts one type into another. === compares without that conversion; == can convert before comparing.

truthiness: Conditions convert values to true or false. A nonempty string is truthy even when its contents say false.

Full new-focus index: values and types; coercion and strict equality; truthiness; functions; control flow; arrays and objects; destructuring and spread; Map and Set; immutability; operators and precedence; strings and numbers; array map filter reduce sort; shallow versus deep copy; complexity analysis; let const var; scope and hoisting; closures; this call bind apply; prototypes and classes; own properties; ES modules; CommonJS; exceptions and finally; promises; async await; timers; race conditions; AbortController; errors; cleanup.

INDIVIDUAL MASTERY OUTCOMES

Neha: Reject invalid job input without silent coercion; compose parser, queue and lookup into one reproducible job flow.

Madhavi: Protect numeric stock rules and show one isolated reservation/submission flow without input mutation.

COACH'S UNSEEN VARIATION

Why does '3' belong at the form boundary but fail the numeric stock boundary?

Progressive hints: start from the contract; reuse the accepted units; trace one invalid input or failure before widening scope.

References: JS-01, JS-02, JS-03 | **PostgreSQL:** No database yet

WEEK 02

JavaScript depth and bounded work

NEHA | Job-side contribution

Add a cancellable, bounded job-prefetch queue and a generator that streams job summaries without materializing every row.

MADHAVI | Inventory-side contribution

Add bounded bulk-stock previews with per-item failure results and weakly held session metadata; integrate exact external IDs without unsafe Number conversion.

WHY THIS WEEK / PRODUCT PROGRESS

Bulk job and stock processing share predictable concurrency, cancellation and cleanup contracts.

CONCEPTS TO UNDERSTAND - BOTH LEARNERS

iterables and generators: An iterable produces an iterator; a generator function can yield values lazily and preserve execution state.

WeakMap and WeakSet: Weak collections reference object keys without keeping them alive solely through that collection; they are not enumerable.

promise combinators: Promise helpers combine multiple operations with different completion and rejection policies.

Full new-focus index: iterables and generators; garbage collection; WeakMap and WeakSet; BigInt and Symbol; event loop; microtasks versus timers; promise combinators; error propagation; concurrency versus parallelism; resource cleanup.

INDIVIDUAL MASTERY OUTCOMES

Neha: Keep job-prefetch concurrency bounded, preserve ordering and cancel without leaking work.

Madhavi: Return per-part bulk results, preserve exact IDs and explain weak references without claiming deterministic garbage collection.

COACH'S UNSEEN VARIATION

One item fails while another is cancelled: which results and resources remain?

Progressive hints: start from the contract; reuse the accepted units; trace one invalid input or failure before widening scope.

References: JS-01, JS-02, JS-04 | **PostgreSQL:** No database yet

WEEK 03

TypeScript fundamentals and domain states

NEHA | Job-side contribution

Type the job queue, legal assignment/completion transitions and discriminated job results.

MADHAVI | Inventory-side contribution

Type inventory/reservation states, quantity/value results and safe stock projections.

WHY THIS WEEK / PRODUCT PROGRESS

Existing JavaScript workflows gain strict typed boundaries without changing their accepted behavior.

CONCEPTS TO UNDERSTAND - BOTH LEARNERS

discriminated unions: A union has a shared tag whose value identifies which object shape applies.

narrowing: Checks refine a broad type into a more specific type within a branch.

never: never describes a value that cannot occur, useful for unreachable and exhaustive branches.

Full new-focus index: type inference; interfaces and type aliases; optional properties; literal types; unions and intersections; discriminated unions; narrowing; never; primitive annotations; array and tuple types; enum versus literal union.

INDIVIDUAL MASTERY OUTCOMES

Neha: Represent legal job states and make an invalid transition a compile-time rejection.

Madhavi: Represent reservation states and separate successful values from explicit failure results.

COACH'S UNSEEN VARIATION

Why can a type annotation still allow bad data arriving from outside the program?

Progressive hints: start from the contract; reuse the accepted units; trace one invalid input or failure before widening scope.

References: TS-01, TS-02 | **PostgreSQL:** No database yet

WEEK 04

TypeScript advanced contracts and runtime validation

NEHA | Job-side contribution

Build typed job pagination/projections and runtime-validated job request/response schemas.

MADHAVI | Inventory-side contribution

Build generic inventory projections and versioned reservation request/receipt schemas with classified errors.

WHY THIS WEEK / PRODUCT PROGRESS

A shared contracts package supports future web, mobile and Node consumers.

CONCEPTS TO UNDERSTAND - BOTH LEARNERS

generics: Generics parameterize a type or function while preserving relationships between input and output types.

unknown versus any: unknown requires checking before use; any bypasses many compiler guarantees.

runtime versus compile-time contracts: Compile-time contracts check source code; runtime contracts check deployed interactions and actual data.

Full new-focus index: generics; constraints; keyof; indexed access; mapped types; conditional types; utility types; structural typing; readonly; function types; overloads; optional and rest parameters; unknown versus any; type guards; runtime schemas; typed errors; null and undefined; serialization; strict compiler options; exhaustiveness; type-only imports; module boundaries; declaration output; type compatibility; runtime versus compile-time contracts; tsconfig; source maps; TypeScript compilation versus execution.

INDIVIDUAL MASTERY OUTCOMES

Neha: Validate a malformed job response at runtime and preserve typed pagination behavior.

Madhavi: Reject an invalid reservation receipt and support a compatible schema version.

COACH'S UNSEEN VARIATION

An old client sends a missing field: reject, default or negotiate, and why?

Progressive hints: start from the contract; reuse the accepted units; trace one invalid input or failure before widening scope.

References: TS-02, TS-03, TS-04 | **PostgreSQL:** No database yet

WEEK 05

Small Node API and light PostgreSQL

NEHA | Job-side contribution

Implement job create/list/read endpoints, the jobs/technicians schema and assignment lookup repository.

MADHAVI | Inventory-side contribution

Implement part create/list/read endpoints, the parts schema and reservation-request intake repository.

WHY THIS WEEK / PRODUCT PROGRESS

The job-read adapter connects to a small API; real jobs, parts and pending reservation requests persist. The commit adapter remains fixture-backed until week 12.

CONCEPTS TO UNDERSTAND - BOTH LEARNERS

HTTP semantics: HTTP methods, status codes, headers, and bodies describe requests and their outcomes.

SQL: SQL queries and changes relational data through declared statements.

authentication versus authorization: Authentication establishes identity; authorization decides whether that identity may perform an action.

Full new-focus index: routing; HTTP semantics; middleware; request lifecycle; async errors; configuration; logging; dependency injection; ESM; headers and cookies; JSON parsing limits; Buffer; filesystem promises; EventEmitter; package scripts and lockfiles; relational modelling; foreign keys; unique and check constraints; SQL; migrations; authentication versus authorization; sessions versus tokens; password hashing when applicable; RBAC and resource rules; tenant isolation; CSRF and CORS; rate limiting; TLS basics; HTTP security headers.

INDIVIDUAL MASTERY OUTCOMES

Neha: Create/read a persisted job with parameterized SQL and appropriate authorization.

Madhavi: Create/read a persisted part and store a pending request without pretending stock was committed.

COACH'S UNSEEN VARIATION

Trace one request from HTTP validation through SQL to a response, including a rejected request.

Progressive hints: start from the contract; reuse the accepted units; trace one invalid input or failure before widening scope.

References: NODE-01, NODE-02, NODE-04 | **PostgreSQL:** Light foundation

WEEK 06

React foundations with real reads

NEHA | Job-side contribution

Build the dispatcher job board, summary components and job-detail route against the API.

MADHAVI | Inventory-side contribution

Build the stock board, part-detail route and cost-preview panel against the API.

WHY THIS WEEK / PRODUCT PROGRESS

Dispatchers can browse persisted jobs and stock in one React shell.

CONCEPTS TO UNDERSTAND - BOTH LEARNERS

state: State is stored information that can change and influence behavior or rendering.

derived state: Derived state can be computed from other inputs instead of separately stored.

reconciliation: Reconciliation resolves differences between local and authoritative state through a defined policy.

Full new-focus index: JSX; components; props; composition; events; state; reconciliation; keys; derived state; immutable updates; Rules of Hooks; useReducer; Context; controlled and uncontrolled inputs; form events; validation; refs; focus; URL state; accessibility; loading and empty states; portals; focus management; indexes.

INDIVIDUAL MASTERY OUTCOMES

Neha: Explain job-board state, keys and component composition; read real API results.

Madhavi: Explain stock-board state and derived pricing without redundant state copies.

COACH'S UNSEEN VARIATION

What breaks when list keys identify positions instead of stable records?

Progressive hints: start from the contract; reuse the accepted units; trace one invalid input or failure before widening scope.

References: REACT-01, REACT-02, NODE-02 | **PostgreSQL:** Everyday product queries

WEEK 07

React forms, effects and failure recovery

NEHA | Job-side contribution

Implement an accessible job-assignment form, URL filters and cancellable job-detail hook.

MADHAVI | Inventory-side contribution

Implement an accessible reservation-request form, part filters and stock-detail hook; isolate panel failures.

WHY THIS WEEK / PRODUCT PROGRESS

Both workflows handle invalid input, focus, loading, empty and failed requests; requests remain pending until the safe reservation service exists.

CONCEPTS TO UNDERSTAND - BOTH LEARNERS

useEffect: An effect synchronizes a component with an external system after rendering.

error boundaries: A boundary renders a fallback for supported descendant render/lifecycle failures; it does not catch every async error.

accessibility: Accessibility makes interactions understandable and usable with different input methods and assistive technology.

Full new-focus index: useEffect; dependency arrays; useRef; custom hooks; StrictMode; render versus commit; component testing; integration testing; mock boundaries; error boundaries; async event errors; render errors; test isolation.

INDIVIDUAL MASTERY OUTCOMES

Neha: Recover an accessible assignment form from validation and async failures with correct focus.

Madhavi: Recover a reservation-request panel without breaking the rest of the interface.

COACH'S UNSEEN VARIATION

Which errors can a render error boundary handle, and which require explicit async handling?

Progressive hints: start from the contract; reuse the accepted units; trace one invalid input or failure before widening scope.

References: REACT-02, REACT-03, REACT-04 | **PostgreSQL:** Everyday product queries

WEEK 08

React server state and optimistic changes

NEHA | Job-side contribution

Add job-list caching, pagination and optimistic assignment with rollback and accessible status feedback.

MADHAVI | Inventory-side contribution

Add stock/request caching, invalidation and optimistic editing of pending requests with rollback.

WHY THIS WEEK / PRODUCT PROGRESS

Both features stay consistent after writes, errors and reordered responses.

CONCEPTS TO UNDERSTAND - BOTH LEARNERS

server state versus client state: Server state is remote authoritative data; client state includes local interaction and unsaved intent.

optimistic state: Optimistic state displays expected success before the server confirms, with a recovery policy.

rollback: Rollback restores a safe earlier application or state version after failure; database reversal requires separate reasoning.

Full new-focus index: server state versus client state; query keys; pagination; freshness; invalidation; request deduplication; optimistic state; mutation ordering; rollback; state machines; accessible status feedback.

INDIVIDUAL MASTERY OUTCOMES

Neha: Show correct job cache invalidation and rollback when assignments resolve out of order.

Madhavi: Keep pending-request edits and stock reads consistent after failure or stale completion.

COACH'S UNSEEN VARIATION

What happens when a second optimistic change succeeds before the first change fails?

Progressive hints: start from the contract; reuse the accepted units; trace one invalid input or failure before widening scope.

References: REACT-05, REACT-06 | **PostgreSQL:** Everyday product queries

WEEK 09

Advanced React rendering and measured performance

NEHA | Job-side contribution

Add lazy job-route loading and an authenticated job-summary rendering experiment; profile and fix dispatch-board rerenders.

MADHAVI | Inventory-side contribution

Add a lazy inventory report route and an authenticated report rendering experiment; profile and fix large stock-list interactions.

WHY THIS WEEK / PRODUCT PROGRESS

Compare client rendering with supported streamed rendering/hydration on both routes and improve one measured bottleneck per feature.

CONCEPTS TO UNDERSTAND - BOTH LEARNERS

hydration: Hydration connects client React behavior to existing server-rendered markup.

memoization: Memoization reuses a result when relevant inputs have not changed.

referential equality: Referential equality compares object identity, not a deep comparison of contents.

Full new-focus index: lazy loading; Suspense; transitions; streaming SSR; hydration; server/client boundaries; framework integration; bundle size; React Profiler; memoization; referential equality; context propagation; virtualization; bundle splitting; memory cleanup; useMemo versus useCallback.

INDIVIDUAL MASTERY OUTCOMES

Neha: Compare job rendering strategies and fix one measured dispatch rerender bottleneck.

Madhavi: Compare report rendering strategies and fix one measured stock-list bottleneck.

COACH'S UNSEEN VARIATION

Show evidence that your optimization improves the workload instead of adding unnecessary caching.

Progressive hints: start from the contract; reuse the accepted units; trace one invalid input or failure before widening scope.

References: REACT-07, REACT-08 | **PostgreSQL:** Everyday product queries

WEEK 10

React Native foundations

NEHA | Job-side contribution

Build assigned-job navigation, adaptive job-detail screens and deep links.

MADHAVI | Inventory-side contribution

Build mobile part lookup, inventory-request navigation and deep links.

WHY THIS WEEK / PRODUCT PROGRESS

One technician app contains both job and inventory features with safe areas, keyboard handling and accessible states.

CONCEPTS TO UNDERSTAND - BOTH LEARNERS

native components: Native components map React Native UI descriptions to platform views.

deep links: A deep link routes an external location to a particular application screen.

safe areas: Safe areas avoid content being obscured by device cutouts and system UI.

Full new-focus index: native components; Flexbox; safe areas; navigation; deep links; keyboard handling; platform differences.

INDIVIDUAL MASTERY OUTCOMES

Neha: Navigate and deep-link to a job with correct safe areas, keyboard and accessibility behavior.

Madhavi: Navigate and deep-link to a part/request with the same platform concerns.

COACH'S UNSEEN VARIATION

An incoming deep link has an invalid ID while the app is already open: how should it recover?

Progressive hints: start from the contract; reuse the accepted units; trace one invalid input or failure before widening scope.

References: RN-01 | **PostgreSQL:** Everyday product queries

WEEK 11

Mobile media, persistence and interaction

NEHA | Job-side contribution

Add completion photos, permission/recovery states and persisted completion drafts; profile the job/evidence list.

MADHAVI | Inventory-side contribution

Add stock evidence photos and persisted request drafts; profile stock/evidence lists and interactions.

WHY THIS WEEK / PRODUCT PROGRESS

Technicians retain drafts and local media across app restarts; the API stores attachment metadata.

CONCEPTS TO UNDERSTAND - BOTH LEARNERS

app lifecycle: The OS can foreground, background, suspend, or terminate a mobile process.

secure versus ordinary storage: Sensitive credentials need protected storage appropriate to the platform; ordinary preferences are not a secret store.

JS and UI threads: JavaScript execution and platform UI work have distinct scheduling concerns.

Full new-focus index: camera and media permissions; local files; local persistence; app lifecycle; async writes; secure versus ordinary storage; FlatList; stable keys; gestures; animations; JS and UI threads; image memory; Hermes profiling.

INDIVIDUAL MASTERY OUTCOMES

Neha: Retain completion drafts/photos after restart and diagnose a measured list or image issue.

Madhavi: Retain stock drafts/photos and recover from denied permission or interrupted persistence.

COACH'S UNSEEN VARIATION

When is ordinary local storage inappropriate, and what happens if a write is interrupted?

Progressive hints: start from the contract; reuse the accepted units; trace one invalid input or failure before widening scope.

References: RN-02, RN-03, NODE-02 | **PostgreSQL:** Everyday product queries

WEEK 12

Node and PostgreSQL correctness: deep focus

NEHA | Job-side contribution

Implement atomic completion that consumes already-reserved parts without a second stock deduction, with authorization and replay-safe results.

MADHAVI | Inventory-side contribution

Implement atomic reservation/release, concurrent-stock guards and replay-safe reservation receipts.

WHY THIS WEEK / PRODUCT PROGRESS

Concurrent completion/reservation cannot overspend stock or duplicate effects; pending requests now commit through protected transactional services.

CONCEPTS TO UNDERSTAND - BOTH LEARNERS

transactions: A transaction groups changes that commit together or are rolled back.

isolation: Isolation determines how concurrent transactions observe and affect one another.

idempotency: An idempotent operation has no extra business effect when repeated according to its contract.

Full new-focus index: transactions; locking; isolation; idempotency; unique constraints; atomicity; request fingerprints; concurrent promises; business-state guards.

INDIVIDUAL MASTERY OUTCOMES

Neha: Complete a job atomically, consume reserved stock once and replay without duplicate effects.

Madhavi: Handle competing reservations with nonnegative stock, atomic release and replay-safe receipts.

COACH'S UNSEEN VARIATION

Stock is 5 and two requests reserve 3 each: state the allowed result before running the scenario.

Progressive hints: start from the contract; reuse the accepted units; trace one invalid input or failure before widening scope.

References: NODE-02, NODE-03, NODE-04 | **PostgreSQL:** Heavy correctness work

WEEK 13

Durable offline queues and native boundaries

NEHA | Job-side contribution

Persist a job-completion operation queue; add a small native attachment-checksum adapter through the jobs/evidence boundary.

MADHAVI | Inventory-side contribution

Persist an inventory-operation queue; add a small native local-file metadata adapter through the inventory/evidence boundary.

WHY THIS WEEK / PRODUCT PROGRESS

Both mobile features preserve pending work across termination; each has a bounded native integration exercise with platform evidence.

CONCEPTS TO UNDERSTAND - BOTH LEARNERS

durability: Durability means acknowledged data survives the failures covered by the storage contract.

JSI concepts: JSI is an interface enabling JavaScript and native code to interact; it is not a guarantee of cheap or safe synchronous work.

Fabric concepts: Fabric is React Native's renderer architecture that coordinates React and native view work.

Full new-focus index: native modules; typed specifications; code generation; JSI concepts; Fabric concepts; render commit mount; threading; native build tools; local transactions; durability; local database migrations.

INDIVIDUAL MASTERY OUTCOMES

Neha: Recover a durable job queue after termination and explain the native checksum adapter's threading boundary.

Madhavi: Recover an inventory queue and explain the native metadata adapter and binary compatibility.

COACH'S UNSEEN VARIATION

Which guarantees come from local transactions, native code and server receipts respectively?

Progressive hints: start from the contract; reuse the accepted units; trace one invalid input or failure before widening scope.

References: SYNC-01, RN-04 | **PostgreSQL:** Offline reconciliation

WEEK 14

Offline conflict resolution and resumable uploads

NEHA | Job-side contribution

Add completion-versus-cancellation conflict handling and resumable completion-photo upload.

MADHAVI | Inventory-side contribution

Add reservation-versus-stock-change conflict handling and resumable stock-evidence upload.

WHY THIS WEEK / PRODUCT PROGRESS

Queued operations retry, preserve per-entity ordering and expose conflicts instead of silently overwriting accepted state.

CONCEPTS TO UNDERSTAND - BOTH LEARNERS

optimistic concurrency: An operation checks that the stored version still matches the version it edited.

clock skew: Clock skew means device timestamps can disagree and cannot alone prove global order.

resumability: Resumability continues from a durable checkpoint after interruption.

Full new-focus index: retry and backoff; jitter; per-entity ordering; batching; cancellation; connectivity signals; optimistic concurrency; versions; conflict detection; domain policy; audit history; clock skew; chunking; checksums; resumability; file streams; resource lifecycle; upload authorization; mobile background limits.

INDIVIDUAL MASTERY OUTCOMES

Neha: Resolve stale completion explicitly and resume a cancelled/restarted evidence upload safely.

Madhavi: Resolve stale stock requests explicitly and resume stock evidence without duplicated effects.

COACH'S UNSEEN VARIATION

The client clock is wrong and an upload response is lost: which identifiers establish truth?

Progressive hints: start from the contract; reuse the accepted units; trace one invalid input or failure before widening scope.

References: SYNC-02, SYNC-03, SYNC-04 | **PostgreSQL:** Offline reconciliation

WEEK 15

Recoverable live events and background work

NEHA | Job-side contribution

Deliver job events with reconnect cursors and a recoverable completion-notification consumer.

MADHAVI | Inventory-side contribution

Deliver stock/reservation events and an inventory-alert consumer with bounded retries/dead-letter handling.

WHY THIS WEEK / PRODUCT PROGRESS

Committed events and background effects recover after disconnects, crashes and duplicate delivery.

CONCEPTS TO UNDERSTAND - BOTH LEARNERS

transactional outbox: An outbox stores side-effect intent in the same database transaction as the business change.

at-least-once processing: At-least-once processing can repeat deliveries, so consumers must tolerate duplicates.

idempotent consumers: An idempotent consumer prevents duplicate deliveries from repeating a business effect.

Full new-focus index: SSE versus WebSockets; ordering; cursors; deduplication; reconnect; authentication; snapshot plus event stream; transactional outbox; at-least-once processing; idempotent consumers; queues; dead letters; retry budgets; distributed transactions.

INDIVIDUAL MASTERY OUTCOMES

Neha: Recover missed job events and duplicate completion notifications without losing accepted work.

Madhavi: Recover stock events and inventory-alert retries, including an exhausted retry budget.

COACH'S UNSEEN VARIATION

A process dies after database commit but before publish: where does recovery start?

Progressive hints: start from the contract; reuse the accepted units; trace one invalid input or failure before widening scope.

References: RT-01, RT-02 | **PostgreSQL:** Reliable delivery and scale

WEEK 16

Node streams, workers and resource limits

NEHA | Job-side contribution

Stream a job/evidence CSV export with cancellation and bounded memory; enforce graceful API shutdown.

MADHAVI | Inventory-side contribution

Add an inventory-reconciliation report worker with a bounded pool and graceful worker shutdown.

WHY THIS WEEK / PRODUCT PROGRESS

Large reports and CPU work preserve API responsiveness and release resources correctly.

CONCEPTS TO UNDERSTAND - BOTH LEARNERS

backpressure: Backpressure slows a producer when its consumer cannot keep up.

worker_threads: Worker threads run JavaScript in separate execution contexts for suitable CPU work.

CPU versus I/O: CPU work calculates; I/O waits for external resources. They require different scaling strategies.

Full new-focus index: Node streams; backpressure; pipeline; async iterators; buffering; database cursors; CSV correctness; event-loop delay; worker_threads; pool sizing; CPU versus I/O; message serialization; signals; graceful shutdown; timeouts.

INDIVIDUAL MASTERY OUTCOMES

Neha: Stream a large export with bounded memory and clean cancellation; shut down the API safely.

Madhavi: Bound worker concurrency and preserve resources when the report worker fails or shuts down.

COACH'S UNSEEN VARIATION

Why does async/await not make CPU-heavy work stop blocking the JavaScript thread?

Progressive hints: start from the contract; reuse the accepted units; trace one invalid input or failure before widening scope.

References: RT-03, RT-04 | **PostgreSQL:** Reliable delivery and scale

WEEK 17

Web micro frontends and mobile feature modules

NEHA | Job-side contribution

Extract dispatch as an independently built web remote; establish jobs/evidence mobile package interfaces.

MADHAVI | Inventory-side contribution

Build the reporting remote and shell integration; establish inventory/sync mobile package interfaces.

WHY THIS WEEK / PRODUCT PROGRESS

Two independently built web features compose through explicit contracts; mobile features remain compatible within the native app release.

CONCEPTS TO UNDERSTAND - BOTH LEARNERS

micro frontends versus modules: Micro frontends compose separately built web features; modules can share one build and release.

shared React identity: The shell and remotes must use compatible React runtime instances so component and Hook behavior remains valid.

mobile release constraints: Mobile changes must respect native binary compatibility, platform requirements and the chosen release/update mechanism.

Full new-focus index: micro frontends versus modules; build-time versus runtime integration; ownership; coupling; shared dependencies; architecture decisions; module federation or equivalent; runtime manifests; shared React identity; bundler configuration; CSS isolation; package boundaries; dependency graphs; navigation contracts; shared domain code; native build compatibility; mobile release constraints.

INDIVIDUAL MASTERY OUTCOMES

Neha: Integrate an independently built dispatch remote and explicit jobs/evidence mobile interfaces.

Madhavi: Integrate reporting and shell artifacts plus inventory/sync mobile interfaces.

COACH'S UNSEEN VARIATION

What cost does runtime integration add for two developers, and when would one modular build be better?

Progressive hints: start from the contract; reuse the accepted units; trace one invalid input or failure before widening scope.

References: MFE-01, MFE-02, MFE-04 | **PostgreSQL:** Reliable delivery and scale

WEEK 18

Micro frontend compatibility and failure recovery

NEHA | Job-side contribution

Add dispatch contract negotiation, remote-load failure fallback and a compatible dispatch release.

MADHAVI | Inventory-side contribution

Add reporting contract negotiation, runtime manifest/cache handling and an independent reporting rollback.

WHY THIS WEEK / PRODUCT PROGRESS

A remote can fail or evolve without breaking the shell; mobile navigation/package contracts survive compatible changes.

CONCEPTS TO UNDERSTAND - BOTH LEARNERS

version negotiation: Version negotiation selects a contract version both communicating components can understand.

runtime manifests: A runtime manifest tells the shell which feature version to load and where its artifact is located.

contract tests: Contract tests check whether a producer and consumer agree on their shared inputs, outputs and errors.

Full new-focus index: event contracts; schema validation; version negotiation; cache invalidation; rollbacks; contract tests.

INDIVIDUAL MASTERY OUTCOMES

Neha: Keep dispatch compatible across a changed contract and show a usable remote-load fallback.

Madhavi: Roll back reporting independently and explain manifest/cache compatibility.

COACH'S UNSEEN VARIATION

The remote loads a second React runtime or a stale contract: how do you diagnose and recover?

Progressive hints: start from the contract; reuse the accepted units; trace one invalid input or failure before widening scope.

References: MFE-03, MFE-04 | **PostgreSQL:** Reliable delivery and scale

WEEK 19

Security and end-to-end performance

NEHA | Job-side contribution

Harden job/evidence authorization and upload limits; measure and fix a dispatch/API/SQL bottleneck.

MADHAVI | Inventory-side contribution

Harden inventory/report authorization and credential handling; measure and fix a stock/report/SQL bottleneck.

WHY THIS WEEK / PRODUCT PROGRESS

Tenant/security boundaries withstand invalid requests, and capacity decisions follow measured query, runtime and device behavior.

CONCEPTS TO UNDERSTAND - BOTH LEARNERS

least privilege: Least privilege grants only the permissions needed for a person, service or database operation.

percentiles: A percentile describes how much of a measured distribution falls below a value; p95 latency includes 95 percent of requests.

query plans: A query plan shows how the database intends to execute a query.

Full new-focus index: query plans; threat modelling; secure storage; input and upload limits; content validation; credential lifecycle; CSRF CORS CSP; least privilege; dependency security; load models; percentiles; profiling; database plans; network cost; memory leaks; capacity and cost.

INDIVIDUAL MASTERY OUTCOMES

Neha: Prove a job/evidence security boundary and improve a measured client/API/SQL bottleneck.

Madhavi: Prove an inventory/report security boundary and improve a measured query or transaction bottleneck.

COACH'S UNSEEN VARIATION

Explain the chosen percentile, workload and query plan before claiming a performance improvement.

Progressive hints: start from the contract; reuse the accepted units; trace one invalid input or failure before widening scope.

References: OPS-01, OPS-02 | **PostgreSQL:** Heavy performance and recovery work

WEEK 20

Observability, deployment and database recovery

NEHA | Job-side contribution

Trace a failed completion across mobile/API/worker; ship a safe completion-service migration and demonstrate restore/recovery.

MADHAVI | Inventory-side contribution

Trace a failed reservation/report; ship a safe inventory migration and demonstrate rollback/recovery.

WHY THIS WEEK / PRODUCT PROGRESS

The team can diagnose a real workflow failure, deploy reproducible artifacts and recover verified PostgreSQL data.

CONCEPTS TO UNDERSTAND - BOTH LEARNERS

AsyncLocalStorage: Node's AsyncLocalStorage associates context with an asynchronous operation, such as its request or correlation ID.

restore verification: Restore verification checks recovered data and business behavior; possessing a backup alone does not prove recovery works.

recovery objectives: Recovery objectives specify acceptable data loss and acceptable time to restore a working service.

Full new-focus index: structured logging; metrics; tracing; AsyncLocalStorage; SLIs and SLOs; alert design; privacy; CI/CD; artifact reproducibility; migration safety; backups; restore verification; recovery objectives; unit versus integration versus end-to-end tests; property-based testing.

INDIVIDUAL MASTERY OUTCOMES

Neha: Trace one failed completion and verify a safe migration plus restored business data.

Madhavi: Trace one failed reservation/report and verify rollback plus recovered stock invariants.

COACH'S UNSEEN VARIATION

A backup restores successfully but records are inconsistent: is recovery complete?

Progressive hints: start from the contract; reuse the accepted units; trace one invalid input or failure before widening scope.

References: OPS-03, OPS-04 | **PostgreSQL:** Heavy performance and recovery work

WEEK 21

Compatible product evolution and disputed work

NEHA | Job-side contribution

Support an older mobile client while adding disputed completion/evidence resolution and a compensating action.

MADHAVI | Inventory-side contribution

Support an older web/client contract while adding disputed reservation resolution and a compensating inventory action.

WHY THIS WEEK / PRODUCT PROGRESS

A domain change preserves older clients, audit history and cross-platform invariants.

CONCEPTS TO UNDERSTAND - BOTH LEARNERS

API compatibility: API compatibility keeps existing clients working when request, response or business behavior changes.

compensating actions: A compensating action creates a new business operation to correct an accepted effect while preserving its history.

domain invariants: Domain invariants are business rules that must remain true, such as accepted consumption never exceeding available stock.

Full new-focus index: API compatibility; domain migrations; local migrations; feature negotiation; deprecation; contract testing; domain invariants; state transitions; compensating actions; UX; audit trails; cross-platform contracts.

INDIVIDUAL MASTERY OUTCOMES

Neha: Preserve an older mobile contract while resolving disputed evidence with a traceable compensating action.

Madhavi: Preserve an older inventory/client contract while resolving a disputed reservation without erasing history.

COACH'S UNSEEN VARIATION

Why might undoing a committed operation require a new business action rather than deleting a record?

Progressive hints: start from the contract; reuse the accepted units; trace one invalid input or failure before widening scope.

References: CAP-01, CAP-02 | **PostgreSQL:** Mastery under change

WEEK 22

Independent mastery and product acceptance

NEHA | Job-side contribution

Fix an unseen inventory/report incident or change, improve that product path and defend its tradeoffs without peer implementation.

MADHAVI | Inventory-side contribution

Fix an unseen job/mobile incident or change, improve that product path and defend its tradeoffs without peer implementation.

WHY THIS WEEK / PRODUCT PROGRESS

Both deliver an accepted change outside their usual feature lane and close every remaining independent concept gap.

CONCEPTS TO UNDERSTAND - BOTH LEARNERS

hypothesis testing: In debugging, hypothesis testing proposes a cause, predicts evidence and runs an experiment to confirm or reject it.

architecture tradeoffs: Architecture tradeoffs compare the benefits and costs of a design under the product's actual constraints.

knowledge transfer: Knowledge transfer enables another person to understand and apply a concept, while checking their independent reasoning.

Full new-focus index: debugging; distributed failure; observability; hypothesis testing; regression testing; incident communication; architecture tradeoffs; system boundaries; cost; communication; fundamental recall; knowledge transfer.

INDIVIDUAL MASTERY OUTCOMES

Neha: Independently diagnose and improve an unfamiliar inventory/report workflow, then close individual concept gaps.

Madhavi: Independently diagnose and improve an unfamiliar job/mobile workflow, then close individual concept gaps.

COACH'S UNSEEN VARIATION

Ask each learner to predict an unseen failure, justify a fix and explain when their decision would change.

Progressive hints: start from the contract; reuse the accepted units; trace one invalid input or failure before widening scope.

References: CAP-03, CAP-04 | **PostgreSQL:** Mastery under change

What mastery means at week 22

Both learners must independently meet the same threshold. The roadmap and this PDF establish planned coverage; running the curriculum validator does not establish learning or product correctness.

UNDERSTANDING

Explain the concept plainly and predict important edge cases without prompting.

IMPLEMENTATION

Implement or adapt correct behavior independently, then integrate it with the product.

DEBUGGING

Locate the cause using observed evidence and a reproducible scenario.

TRADEOFFS

Defend alternatives and state what evidence would change the decision.

TRANSFER

Solve an unseen variation and teach it accurately in another feature context.

Scoring: 0 = cannot demonstrate; 1 = needs assistance; 2 = independent; 3 = adapts/generalizes. Pass requires at least 2 in every dimension. Correctness or security failures block the gate.

THE ACCEPTED PRODUCT

A working web dispatcher and mobile technician workflow, correct persisted jobs/inventory, durable offline behavior, recoverable events, bounded reports/workers and independently built web features. Both can verify migration, rollback and database restore behavior.

THE INDIVIDUAL COVERAGE AUDIT

All 338 concepts are independently verified in each learner's ledger. Each fixes an unseen issue outside their usual feature lane. Retention gaps receive a precise product slice or bounded experiment; the schedule extends rather than claiming completion.

REVIEWER AND COACH RESPONSIBILITIES

Prem-Tomar reviews and is the only merger to master. Before seeing code, the coach states the expected outcome and assumptions, reproduces the scenario and revises mistaken expectations. PR/session evidence is enough; a separate formal report is not required.

No new issues or automation are authorized by this PDF. The 14 existing week-one issues are assigned seven each. Future frontend tasks include screen images; test-writing suggestions remain optional PR feedback.

Every concept, for both learners

Definitions are brief starting points. A first-focus week is a planning cue, not a mastery claim; later features revisit the concept. Both learners must explain, predict, implement/adapt and transfer it independently.

001 values and types (W01)
A value is data; its type determines which operations make sense. JavaScript has primitive values and objects.
Example: 3 is a number; '3' is a string; {available:3} is an object.

002 coercion and strict equality (W01)
Coercion converts one type into another. === compares without that conversion; == can convert before comparing.
Example: '0' == 0 is true, but '0' === 0 is false.

003 truthiness (W01)
Conditions convert values to true or false. A nonempty string is truthy even when its contents say false.
Example: Boolean('false') is true; Boolean('') is false.

004 functions (W01)
A function is reusable behavior with parameters, a return value, and possibly side effects.
Example: validateQuantity(3) returns a result; it should not change outside state.

005 control flow (W01)
Control flow determines which statements execute and in what order, using branches, loops, and early returns.
Example: Reject an invalid quantity before reserving stock.

006 arrays and objects (W01)
Arrays are ordered collections; objects associate property names with values.

007 destructuring and spread (W01)
Destructuring extracts fields; spread copies enumerable properties or expands an iterable. Object spread is shallow.

008 Map and Set (W01)
Map stores key/value associations; Set stores unique values. They suit lookup and duplicate detection.

009 immutability (W01)
Treat existing data as unchanged and create new values when updating it.

010 operators and precedence (W01)
Operators perform calculations or comparisons. Precedence decides grouping when parentheses are absent.
*Example: 2 + 3 * 4 evaluates to 14; (2 + 3) * 4 evaluates to 20.*

011 strings and numbers (W01)
Strings represent text; numbers represent numeric values. Their operators and conversions can behave differently.
Example: '2' + 3 produces '23'; 2 + 3 produces 5.

012 array map filter reduce sort (W01)
map transforms items, filter selects items, reduce combines them, and sort orders them; sort mutates its array.

013 shallow versus deep copy (W01)
A shallow copy shares nested references; a deep copy duplicates supported nested data.

014 complexity analysis (W01)
Describe how time and memory requirements grow with input size, rather than relying only on one timing.

015 let const var (W01)
let and const are block scoped; var is function scoped. const prevents rebinding, not object mutation.

016 scope and hoisting (W01)
Scope controls where bindings can be accessed. Bindings are created before execution, but initialization rules differ.

017 closures (W01)
A function can retain access to bindings from the scope where it was created.

018 this call bind apply (W01)
this depends on invocation for ordinary functions; call/apply supply it, and bind creates a bound function.

019 prototypes and classes (W01)
Objects can inherit property lookup through prototypes. Class syntax builds on this object model.

020 own properties (W01)
Own properties exist directly on an object, rather than being inherited from its prototype.

021 ES modules (W01)
JavaScript modules use import/export with explicit dependencies and live exported bindings.

022 CommonJS (W01)
Node's CommonJS module system uses require and module.exports; interoperability depends on runtime rules.

023 exceptions and finally (W01)
throw interrupts normal control flow; catch handles errors; finally runs cleanup when the try block exits.

024 iterables and generators (W02)
An iterable produces an iterator; a generator function can yield values lazily and preserve execution state.

025 garbage collection (W02)
The runtime can reclaim objects that are no longer reachable. Exact collection timing is not guaranteed.

026 WeakMap and WeakSet (W02)
Weak collections reference object keys without keeping them alive solely through that collection; they are not enumerable.

027 BigInt and Symbol (W02)
BigInt represents integers beyond ordinary number precision; Symbol creates unique identity values.

028 promises (W01)
A Promise represents an eventual fulfilled value or rejection; callbacks run after the current synchronous work.

Both learners | First planned focus shown as W01-W22

029 async await (W01)
async returns a Promise; await pauses that function's continuation while other work can proceed.

030 timers (W01)
Timers schedule callbacks after a delay threshold; execution can be later when the runtime is busy.

031 race conditions (W01)
An outcome depends on timing or ordering between operations that overlap.

032 AbortController (W01)
An AbortController sends a cancellation signal to APIs that support it; cancellation remains cooperative.

033 errors (W01)
Errors describe failed operations; programs need a deliberate policy for reporting and recovering from them.

034 cleanup (W01)
Cleanup releases resources your operation owns when it completes, fails, or is abandoned.

035 event loop (W02)
The event loop coordinates execution of scheduled callbacks around synchronous JavaScript work.

036 microtasks versus timers (W02)
Promise reactions run as microtasks; timer callbacks run through later scheduled tasks. Runtime details still matter.

037 promise combinators (W02)
Promise helpers combine multiple operations with different completion and rejection policies.

038 error propagation (W02)
A thrown error or rejected Promise must reach a handler that owns the failure policy.

039 concurrency versus parallelism (W02)
Concurrency overlaps progress; parallelism executes work simultaneously. Async I/O does not itself imply multiple CPU threads.

040 resource cleanup (W02)
Cleanup releases resources your operation owns when it completes, fails, or is abandoned.

041 type inference (W03)
TypeScript often derives a value's type from its expression and context.

042 interfaces and type aliases (W03)
Both describe types; aliases can also name unions and other type expressions.

043 optional properties (W03)
An optional property may be absent; absence is not always interchangeable with a supplied undefined.

044 literal types (W03)
A literal type allows one exact value rather than every value of its primitive type.

045 unions and intersections (W03)
A union accepts one of several shapes; an intersection combines requirements.

046 discriminated unions (W03)
A union has a shared tag whose value identifies which object shape applies.

047 narrowing (W03)
Checks refine a broad type into a more specific type within a branch.

048 never (W03)
never describes a value that cannot occur, useful for unreachable and exhaustive branches.

049 primitive annotations (W03)
TypeScript annotations describe allowed primitive values at compile time.

050 array and tuple types (W03)
An array repeats an element type; a tuple describes specific positions and their types.

051 enum versus literal union (W03)
An enum names related constants; a literal union limits values to specific alternatives.

052 generics (W04)
Generics parameterize a type or function while preserving relationships between input and output types.

053 constraints (W04)
A generic constraint limits which types a parameter accepts.

054 keyof (W04)
keyof produces the union of a type's property names.

055 indexed access (W04)
Indexed access extracts a property or collection element type from another type.

056 mapped types (W04)
A mapped type transforms properties according to a rule.

057 conditional types (W04)
A conditional type selects a type result based on a type relationship.

058 utility types (W04)
Built-in helpers transform types for common needs.

059 structural typing (W04)
Compatibility depends largely on shape rather than a type's declared name.

060 readonly (W04)
readonly prevents certain mutations through a TypeScript reference; it does not freeze runtime objects.

061 function types (W04)
A function type specifies parameters and return type, including callback contracts.

Both learners | First planned focus shown as W01-W22

062 overloads (W04)
Overloads describe distinct supported calling shapes while sharing one implementation.

063 optional and rest parameters (W04)
Optional parameters can be omitted; rest parameters collect a variable number of arguments.

064 unknown versus any (W04)
unknown requires checking before use; any bypasses many compiler guarantees.

065 type guards (W04)
A predicate or checked condition narrows a type by establishing a property.

066 runtime schemas (W04)
A runtime schema checks actual data shapes; static types alone do not inspect incoming JSON.

067 typed errors (W04)
Explicit error shapes make failure handling predictable without relying on message text.

068 null and undefined (W04)
null commonly represents an explicit empty value; undefined often represents absence or an uninitialized result.

069 serialization (W04)
Serialization converts values into a representation another process or storage can consume.

070 strict compiler options (W04)
tsconfig controls compilation and checks; strict options expose unsafe assumptions.

071 exhaustiveness (W04)
Every declared variant must be handled; adding a variant should expose missing decisions.

072 type-only imports (W04)
Type-only imports carry compile-time information and can be removed from emitted JavaScript.

073 module boundaries (W04)
Public interfaces separate packages; consumers use exported behavior without depending on a module's internal details.

074 declaration output (W04)
Declaration files publish type information for consumers of compiled packages.

075 type compatibility (W04)
A type is compatible when it can be used safely where another type is expected under compiler rules.

076 runtime versus compile-time contracts (W04)
Compile-time contracts check source code; runtime contracts check deployed interactions and actual data.

077 tsconfig (W04)
tsconfig controls compilation and checks; strict options expose unsafe assumptions.

078 source maps (W04)
Source maps relate generated code locations to original source for debugging.

079 TypeScript compilation versus execution (W04)
TypeScript checks/transforms source; the runtime executes JavaScript and does not automatically enforce annotations.

080 JSX (W06)
JSX describes UI elements using syntax transformed into JavaScript; it is not an HTML string.

081 components (W06)
Components package UI behavior and presentation into reusable units.

082 props (W06)
Props are inputs passed from a parent to a component.

083 composition (W06)
Composition builds larger behavior by connecting smaller units through explicit interfaces.

084 events (W06)
Events represent user or system actions; handlers decide how state changes.

085 state (W06)
State is stored information that can change and influence behavior or rendering.

086 reconciliation (W06)
Reconciliation resolves differences between local and authoritative state through a defined policy.

087 keys (W06)
Keys identify siblings across list changes; stable domain IDs avoid mismatched identity.

088 derived state (W06)
Derived state can be computed from other inputs instead of separately stored.

089 immutable updates (W06)
Treat existing data as unchanged and create new values when updating it.

090 Rules of Hooks (W06)
Hooks follow stable calling order and permitted call locations so React can associate state correctly.

091 useReducer (W06)
useReducer expresses state changes through actions and a reducer function.

092 Context (W06)
Context distributes values through a component tree without passing each intermediate prop.

093 controlled and uncontrolled inputs (W06)
Controlled inputs take their current value from React state; uncontrolled inputs retain it in the host input.

094 form events (W06)
Events represent user or system actions; handlers decide how state changes.

Both learners | First planned focus shown as W01-W22

095 validation (W06)
Validation checks incoming data against explicit rules before business code accepts or changes it.

096 refs (W06)
A ref retains a mutable value across renders without its changes causing a render.

097 focus (W06)
Focus determines where keyboard input goes; intentional movement makes recovery and navigation usable.

098 routing (W05)
Routing selects an endpoint; middleware applies cross-cutting request behavior around handlers.

099 URL state (W06)
URL state represents shareable navigation/filter values in the address.

100 accessibility (W06)
Accessibility makes interactions understandable and usable with different input methods and assistive technology.

101 loading and empty states (W06)
Loading means data is pending; empty means loading completed without matching data.

102 portals (W06)
A portal renders UI into another host container while retaining its React relationship.

103 focus management (W06)
Focus determines where keyboard input goes; intentional movement makes recovery and navigation usable.

104 useEffect (W07)
An effect synchronizes a component with an external system after rendering.

105 dependency arrays (W07)
Effect dependencies describe reactive values used by the effect and when synchronization changes.

106 useRef (W07)
A ref retains a mutable value across renders without its changes causing a render.

107 custom hooks (W07)
A custom hook extracts reusable React behavior while retaining normal hook rules.

108 StrictMode (W07)
StrictMode adds development checks that expose unsafe behavior, including effect lifecycle problems.

109 render versus commit (W07)
Rendering calculates UI; committing applies an accepted result to the host environment.

110 component testing (W07)
Component testing exercises a UI unit's observable behavior with controlled inputs.

111 integration testing (W07)
Integration testing exercises connected parts and their boundaries.

112 mock boundaries (W07)
A mock replaces a dependency; the chosen boundary determines which real behavior is verified.

113 error boundaries (W07)
A boundary renders a fallback for supported descendant render/lifecycle failures; it does not catch every async error.

114 async event errors (W07)
An async event operation can reject outside render-error boundary handling.

115 render errors (W07)
Render errors occur while calculating or processing a component's UI.

116 test isolation (W07)
Isolation prevents one verification scenario's state from contaminating another.

117 server state versus client state (W08)
Server state is remote authoritative data; client state includes local interaction and unsaved intent.

118 query keys (W08)
Query keys identify which remote result is cached and must include relevant inputs.

119 pagination (W08)
Pagination divides a large result into ordered portions with a continuation strategy.

120 freshness (W08)
Freshness describes how current cached data must be for a use case.

121 invalidation (W08)
Invalidation marks cached data as outdated after a relevant change.

122 request deduplication (W08)
Deduplication shares identical in-flight work rather than issuing redundant requests.

123 optimistic state (W08)
Optimistic state displays expected success before the server confirms, with a recovery policy.

124 mutation ordering (W08)
Mutation ordering decides which overlapping changes may update confirmed or displayed state.

125 rollback (W08)
Rollback restores a safe earlier application or state version after failure; database reversal requires separate reasoning.

126 state machines (W08)
A state machine defines allowed states and transitions rather than scattered flags.

Both learners | First planned focus shown as W01-W22

127 accessible status feedback (W08)
Status feedback must be perceivable without depending only on color or sight.

128 lazy loading (W09)
Lazy loading delays acquiring code or data until it is needed.

129 Suspense (W09)
Suspense coordinates fallback UI for supported suspending work through React/framework integration.

130 transitions (W09)
Transitions mark suitable React updates as lower priority so urgent interaction can proceed.

131 streaming SSR (W09)
Streaming server rendering sends useful HTML progressively rather than waiting for all sections.

132 hydration (W09)
Hydration connects client React behavior to existing server-rendered markup.

133 server/client boundaries (W09)
Boundaries decide which work runs on the server and which requires browser interaction.

134 framework integration (W09)
A framework supplies routing, rendering, and data-loading conventions around React.

135 bundle size (W09)
Bundle size affects transfer, parsing, and startup cost.

136 React Profiler (W09)
Profiling measures where time and resources go so changes address actual bottlenecks.

137 memoization (W09)
Memoization reuses a result when relevant inputs have not changed.

138 referential equality (W09)
Referential equality compares object identity, not a deep comparison of contents.

139 context propagation (W09)
Changes in a context value can update consumers; placement and identity influence the scope.

140 virtualization (W09)
Virtualization renders a visible window of a large collection rather than every item.

141 bundle splitting (W09)
Bundle splitting divides code into loadable chunks.

142 memory cleanup (W09)
Cleanup releases resources your operation owns when it completes, fails, or is abandoned.

143 useMemo versus useCallback (W09)
useMemo caches a calculated value; useCallback retains a function identity between relevant changes.

144 native components (W10)
Native components map React Native UI descriptions to platform views.

145 Flexbox (W10)
Flexbox lays out items within a container using direction, alignment, and size rules.

146 safe areas (W10)
Safe areas avoid content being obscured by device cutouts and system UI.

147 navigation (W10)
Navigation manages screen transitions and their history.

148 deep links (W10)
A deep link routes an external location to a particular application screen.

149 keyboard handling (W10)
Keyboard handling keeps fields and actions usable when the software keyboard changes available space.

150 platform differences (W10)
Android and iOS can differ in permission, layout, navigation, and lifecycle behavior.

151 camera and media permissions (W11)
Apps must request and handle platform access to capture or select media.

152 local files (W11)
Local files have storage locations and lifetimes; temporary files may not be durable evidence.

153 local persistence (W11)
Persistence stores data beyond the current process's memory.

154 app lifecycle (W11)
The OS can foreground, background, suspend, or terminate a mobile process.

155 async writes (W11)
An asynchronous write finishes later; acknowledgement must reflect actual durable completion.

156 secure versus ordinary storage (W11)
Sensitive credentials need protected storage appropriate to the platform; ordinary preferences are not a secret store.

157 FlatList (W11)
FlatList is a React Native component for efficiently presenting scrollable collections.

158 stable keys (W11)
Keys identify siblings across list changes; stable domain IDs avoid mismatched identity.

159 gestures (W11)
Gestures translate touch movement into interactions with cancellation and conflict rules.

160 animations (W11)
Animations change visual properties over time and must respect responsiveness and user preferences.

Both learners | First planned focus shown as W01-W22

161 JS and UI threads (W11)
JavaScript execution and platform UI work have distinct scheduling concerns.

162 image memory (W11)
Decoded image memory can exceed compressed download size.

163 Hermes profiling (W11)
Hermes profiling inspects JavaScript work in React Native's JavaScript engine.

164 native modules (W13)
Native modules expose platform functionality through a JavaScript-accessible interface.

165 typed specifications (W13)
A typed native specification describes the interface shared across language boundaries.

166 code generation (W13)
Code generation produces integration code from a declared interface or schema.

167 JSI concepts (W13)
JSI is an interface enabling JavaScript and native code to interact; it is not a guarantee of cheap or safe synchronous work.

168 Fabric concepts (W13)
Fabric is React Native's renderer architecture that coordinates React and native view work.

169 render commit mount (W13)
React Native rendering calculates a tree, commits an accepted result, then mounts host changes.

170 threading (W13)
Threading determines where work executes and how access is coordinated.

171 native build tools (W13)
Native build tools compile and package platform code and dependencies.

172 HTTP semantics (W05)
HTTP methods, status codes, headers, and bodies describe requests and their outcomes.

173 middleware (W05)
Routing selects an endpoint; middleware applies cross-cutting request behavior around handlers.

174 request lifecycle (W05)
A request passes through admission, handling, response, and cleanup, including disconnects.

175 async errors (W05)
A thrown error or rejected Promise must reach a handler that owns the failure policy.

176 configuration (W05)
Configuration supplies environment-specific behavior without hardcoding secrets.

177 logging (W05)
Logging records diagnostic events with relevant context.

178 dependency injection (W05)
Dependency injection supplies collaborators through interfaces rather than hardcoded global imports.

179 ESM (W05)
JavaScript modules use import/export with explicit dependencies and live exported bindings.

180 headers and cookies (W05)
Headers carry request/response metadata; cookies can store browser session identifiers with access rules.

181 JSON parsing limits (W05)
Parsing limits bound untrusted body size and complexity before expensive processing.

182 Buffer (W05)
A Buffer represents binary bytes in Node.js.

183 filesystem promises (W05)
Promise-based filesystem APIs provide asynchronous file operations with explicit errors.

184 EventEmitter (W05)
EventEmitter lets producers emit named events and listeners subscribe; listener lifetime needs management.

185 package scripts and lockfiles (W05)
Scripts name repeatable commands; lockfiles record resolved dependency versions.

186 relational modelling (W05)
Relational modelling organizes entities and relationships into constrained tables.

187 foreign keys (W05)
A foreign key constrains a reference to a valid related record.

188 unique and check constraints (W05)
Database constraints enforce rules even under concurrent application access.

189 SQL (W05)
SQL queries and changes relational data through declared statements.

190 indexes (W06)
Indexes organize data to accelerate selected access patterns, at a write/storage cost.

191 transactions (W12)
A transaction groups changes that commit together or are rolled back.

192 locking (W12)
Locks coordinate access when concurrent work would violate a rule.

193 isolation (W12)
Isolation determines how concurrent transactions observe and affect one another.

Both learners | First planned focus shown as W01-W22

194 migrations (W05)
Migrations change stored schema or data in a controlled, versioned way.

195 query plans (W19)
A query plan shows how the database intends to execute a query.

196 idempotency (W12)
An idempotent operation has no extra business effect when repeated according to its contract.

197 unique constraints (W12)
Database constraints enforce rules even under concurrent application access.

198 atomicity (W12)
Atomicity means grouped changes succeed together or do not take effect.

199 request fingerprints (W12)
A fingerprint identifies the relevant contents of a request for repeat comparison.

200 concurrent promises (W12)
Multiple Promise-producing operations can be in flight at once, requiring bounded admission and error handling.

201 business-state guards (W12)
State guards reject operations that are not legal for the current business state.

202 authentication versus authorization (W05)
Authentication establishes identity; authorization decides whether that identity may perform an action.

203 sessions versus tokens (W05)
Sessions and tokens are ways to carry identity; expiration, storage, and revocation depend on the design.

204 password hashing when applicable (W05)
Password hashing stores a one-way, deliberately expensive verifier instead of a recoverable password.

205 RBAC and resource rules (W05)
Role permissions describe categories of actions; resource rules check the particular object's ownership or assignment.

206 tenant isolation (W05)
Tenant isolation prevents one organization's data from leaking into another's operations.

207 CSRF and CORS (W05)
CSRF abuses ambient credentials for unwanted actions; CORS controls browser cross-origin response access, not API authorization.

208 rate limiting (W05)
Rate limiting bounds how often callers can consume a service.

209 TLS basics (W05)
TLS protects transport confidentiality and integrity and authenticates endpoints through certificate validation.

210 HTTP security headers (W05)
Security headers tell browsers about content, framing, transport, and related restrictions.

211 local transactions (W13)
Local transactions commit related offline records together.

212 durability (W13)
Durability means acknowledged data survives the failures covered by the storage contract.

213 local database migrations (W13)
Local migrations evolve device storage while preserving supported old data.

214 retry and backoff (W14)
Retry transient failures with increasing delays and a bounded stopping policy.

215 jitter (W14)
Jitter adds randomized delay so many clients do not retry at the same instant.

216 per-entity ordering (W14)
Ordering is preserved for operations that depend on the same entity while independent entities can proceed.

217 batching (W14)
Batching combines work to reduce overhead while preserving limits and error visibility.

218 cancellation (W14)
Cancellation defines how unfinished work stops and how owned resources are released.

219 connectivity signals (W14)
A device connection signal is a hint, not proof that a particular service is reachable.

220 optimistic concurrency (W14)
An operation checks that the stored version still matches the version it edited.

221 versions (W14)
Versions identify revisions of data or contracts so changes can be detected and negotiated.

222 conflict detection (W14)
Conflict detection recognizes incompatible concurrent intent rather than silently overwriting it.

223 domain policy (W14)
Domain policy states the business decision when several technically possible behaviors exist.

224 audit history (W14)
An audit history preserves attributable changes and decisions for investigation.

225 clock skew (W14)
Clock skew means device timestamps can disagree and cannot alone prove global order.

226 chunking (W14)
Chunking divides a large payload into manageable pieces.

Both learners | First planned focus shown as W01-W22

227 checksums (W14)
A checksum summarizes content so unexpected changes can be detected under its stated guarantees.

228 resumability (W14)
Resumability continues from a durable checkpoint after interruption.

229 file streams (W14)
Streams process data incrementally instead of loading the entire payload at once.

230 resource lifecycle (W14)
A resource has creation, ownership, use, completion, and cleanup rules.

231 upload authorization (W14)
Upload authorization checks who may create, continue, and attach a file to a resource.

232 mobile background limits (W14)
The OS can limit or stop background execution; clients must not rely on uninterrupted work.

233 SSE versus WebSockets (W15)
SSE sends a server-to-client event stream; WebSockets provide a bidirectional connection with different protocol needs.

234 ordering (W15)
Ordering defines which sequence guarantees consumers can rely on.

235 cursors (W15)
A cursor identifies a continuation position in results or event history.

236 deduplication (W15)
Deduplication recognizes repeated work or events and avoids repeated effects.

237 reconnect (W15)
Reconnect restores a broken connection and may require application-state recovery.

238 authentication (W15)
Authentication establishes identity; authorization decides whether that identity may perform an action.

239 snapshot plus event stream (W15)
A snapshot gives current state; an event stream carries changes. Their boundary must avoid a gap.

240 transactional outbox (W15)
An outbox stores side-effect intent in the same database transaction as the business change.

241 at-least-once processing (W15)
At-least-once processing can repeat deliveries, so consumers must tolerate duplicates.

242 idempotent consumers (W15)
An idempotent consumer prevents duplicate deliveries from repeating a business effect.

243 queues (W15)
Queues buffer work for later consumers and need admission, capacity, and failure policies.

244 dead letters (W15)
Dead letters retain work that cannot be processed normally for inspection or recovery.

245 retry budgets (W15)
A retry budget bounds attempts or time to avoid endless overload.

246 distributed transactions (W15)
Distributed transactions coordinate changes across multiple systems; simpler local transactions do not cover remote effects.

247 Node streams (W16)
Streams process data incrementally instead of loading the entire payload at once.

248 backpressure (W16)
Backpressure slows a producer when its consumer cannot keep up.

249 pipeline (W16)
A pipeline connects processing stages with error and cleanup coordination.

250 async iterators (W16)
Async iterators yield values over time through an asynchronous iteration protocol.

251 buffering (W16)
Buffering temporarily accumulates data; uncontrolled buffering can exhaust memory.

252 database cursors (W16)
A database cursor incrementally reads query results while owning database resources.

253 CSV correctness (W16)
CSV output needs proper quoting, line handling, encoding, and a policy for spreadsheet interpretation.

254 event-loop delay (W16)
Event-loop delay measures how long scheduled JS work is delayed by runtime load.

255 worker_threads (W16)
Worker threads run JavaScript in separate execution contexts for suitable CPU work.

256 pool sizing (W16)
Pool sizing balances concurrency against CPU, memory, and queue overhead.

257 CPU versus I/O (W16)
CPU work calculates; I/O waits for external resources. They require different scaling strategies.

258 message serialization (W16)
Serialization converts values into a representation another process or storage can consume.

Both learners | First planned focus shown as W01-W22

259 signals (W16)
Process signals notify software about lifecycle or other operating-system events.

260 graceful shutdown (W16)
Graceful shutdown stops new work and safely finishes or persists accepted work before exiting.

261 timeouts (W16)
A timeout bounds waiting but may leave an uncertain outcome if work committed elsewhere.

262 micro frontends versus modules (W17)
Micro frontends compose separately built web features; modules can share one build and release.

263 build-time versus runtime integration (W17)
Build-time integration bundles features together; runtime integration loads or connects separately built features during execution.

264 ownership (W17)
Ownership identifies who maintains a feature, its public contracts and its recovery behavior.

265 coupling (W17)
Coupling is how strongly one component depends on another; stronger dependencies make independent change harder.

266 shared dependencies (W17)
Shared dependencies are libraries used by multiple features; their versions and runtime instances must remain compatible.

267 architecture decisions (W17)
An architecture decision explains a system choice, alternatives, evidence and consequences for future change.

268 module federation or equivalent (W17)
Module Federation lets separately built applications share code at runtime; alternatives must meet the same integration requirements.

269 runtime manifests (W17)
A runtime manifest tells the shell which feature version to load and where its artifact is located.

270 shared React identity (W17)
The shell and remotes must use compatible React runtime instances so component and Hook behavior remains valid.

271 bundler configuration (W17)
Bundler settings determine how source code becomes executable artifacts, including imports, chunks and shared libraries.

272 CSS isolation (W17)
CSS isolation prevents a feature's styles from accidentally changing another feature's interface.

273 event contracts (W18)
An event contract defines a message's name, payload, version and meaning for producers and consumers.

274 schema validation (W18)
A runtime schema checks actual data shapes; static types alone do not inspect incoming JSON.

275 version negotiation (W18)
Version negotiation selects a contract version both communicating components can understand.

276 cache invalidation (W18)
Invalidation marks cached data as outdated after a relevant change.

277 rollbacks (W18)
Rollback restores a safe earlier application or state version after failure; database reversal requires separate reasoning.

278 contract tests (W18)
Contract tests check whether a producer and consumer agree on their shared inputs, outputs and errors.

279 package boundaries (W17)
Package boundaries define public imports and prevent consumers from relying on private implementation details.

280 dependency graphs (W17)
A dependency graph shows which packages rely on others; cycles and hidden dependencies make change harder.

281 navigation contracts (W17)
Navigation contracts define route names, parameters and expected destination behavior across mobile features.

282 shared domain code (W17)
Shared domain code contains platform-independent business rules reused by web, mobile and server features.

283 native build compatibility (W17)
Native build compatibility means feature code and native dependencies work with the application's platform toolchain and binary.

284 mobile release constraints (W17)
Mobile changes must respect native binary compatibility, platform requirements and the chosen release/update mechanism.

285 threat modelling (W19)
Threat modelling identifies assets, possible attackers and abuse paths so protections address actual product risks.

286 secure storage (W19)
Sensitive credentials need protected storage appropriate to the platform; ordinary preferences are not a secret store.

287 input and upload limits (W19)
Size, count and format limits bound incoming data so requests cannot exhaust resources or bypass rules.

288 content validation (W19)
Content validation inspects the actual data or file, rather than trusting its name or declared type.

289 credential lifecycle (W19)
Credential lifecycle covers creation, storage, use, rotation, expiry and revocation of secrets or access credentials.

290 CSRF CORS CSP (W19)
CSRF protects authenticated actions, CORS governs browser cross-origin access, and CSP limits allowed content execution.

Both learners | First planned focus shown as W01-W22

291 least privilege (W19)
Least privilege grants only the permissions needed for a person, service or database operation.

292 dependency security (W19)
Dependency security examines third-party packages, known vulnerabilities and update risks in the application's dependencies.

293 load models (W19)
A load model describes realistic request rates, users and workloads for a performance measurement.

294 percentiles (W19)
A percentile describes how much of a measured distribution falls below a value; p95 latency includes 95 percent of requests.

295 profiling (W19)
Profiling measures where time and resources go so changes address actual bottlenecks.

296 database plans (W19)
A query plan shows how the database intends to execute a query.

297 network cost (W19)
Network cost includes transferred bytes, request overhead and delay between the client and server.

298 memory leaks (W19)
A memory leak retains data that is no longer needed, increasing memory use as the application runs.

299 capacity and cost (W19)
Capacity and cost relate required throughput, resource limits and operating expense under a defined workload.

300 structured logging (W20)
Structured logging records consistent named fields so events can be searched and compared reliably.

301 metrics (W20)
Metrics are numeric measurements over time, such as request count, latency or queue depth.

302 tracing (W20)
Tracing connects the steps of one operation across components to show where time or failure occurred.

303 AsyncLocalStorage (W20)
Node's AsyncLocalStorage associates context with an asynchronous operation, such as its request or correlation ID.

304 SLIs and SLOs (W20)
A service-level indicator measures behavior; a service-level objective sets a target for that measurement.

305 alert design (W20)
Alert design identifies a meaningful failure condition, threshold and response without generating unnecessary noise.

306 privacy (W20)
Privacy limits collection, use and exposure of personal or sensitive data, including logs and reports.

307 CI/CD (W20)
Continuous integration validates changes; continuous delivery or deployment prepares or releases reproducible application artifacts.

308 artifact reproducibility (W20)
Artifact reproducibility means the same declared source, dependencies and build settings produce a repeatable release.

309 migration safety (W20)
Migration safety preserves valid data and compatible clients while changing database structures or stored values.

310 backups (W20)
A backup is a preserved copy of data that can be used to recover from loss or corruption.

311 restore verification (W20)
Restore verification checks recovered data and business behavior; possessing a backup alone does not prove recovery works.

312 recovery objectives (W20)
Recovery objectives specify acceptable data loss and acceptable time to restore a working service.

313 unit versus integration versus end-to-end tests (W20)
Unit checks isolate a small behavior; integration checks connected parts; end-to-end checks exercise a complete user workflow.

314 property-based testing (W20)
Property-based testing generates varied inputs and checks general rules, such as stock never becoming negative.

315 API compatibility (W21)
API compatibility keeps existing clients working when request, response or business behavior changes.

316 domain migrations (W21)
Migrations change stored schema or data in a controlled, versioned way.

317 local migrations (W21)
Local migrations evolve device storage while preserving supported old data.

318 feature negotiation (W21)
Feature negotiation determines which capabilities a client and server both support before using them.

319 deprecation (W21)
Deprecation announces that a capability will be retired and gives consumers a supported transition path.

320 contract testing (W21)
Contract testing verifies producer and consumer agreement at a versioned boundary.

321 domain invariants (W21)
Domain invariants are business rules that must remain true, such as accepted consumption never exceeding available stock.

322 state transitions (W21)
A state transition changes an entity from one valid state to another according to business rules.

Both learners | First planned focus shown as W01-W22

323 compensating actions (W21)
A compensating action creates a new business operation to correct an accepted effect while preserving its history.

324 UX (W21)
User experience includes how clearly a person can understand, complete and recover from a product workflow.

325 audit trails (W21)
An audit history preserves attributable changes and decisions for investigation.

326 cross-platform contracts (W21)
Cross-platform contracts preserve business meaning across web, mobile and server implementations.

327 debugging (W22)
Debugging uses observations and controlled experiments to identify the cause of incorrect behavior.

328 distributed failure (W22)
Distributed failure occurs when connected components fail or disagree independently, including partial success and lost responses.

329 observability (W22)
Observability uses logs, metrics and traces to understand a system's internal behavior from its outputs.

330 hypothesis testing (W22)
In debugging, hypothesis testing proposes a cause, predicts evidence and runs an experiment to confirm or reject it.

331 regression testing (W22)
Regression testing checks that a change has not broken behavior that previously worked.

332 incident communication (W22)
Incident communication describes the observed impact, evidence, response and recovery status clearly to the relevant people.

333 architecture tradeoffs (W22)
Architecture tradeoffs compare the benefits and costs of a design under the product's actual constraints.

334 system boundaries (W22)
System boundaries identify responsibilities and interfaces between components, including where failures and trust assumptions change.

335 cost (W22)
Cost includes implementation effort, operating resources, maintenance burden and the effect of failure.

336 communication (W22)
Communication expresses a problem, evidence, decision and expected behavior clearly enough for others to act.

337 fundamental recall (W22)
Fundamental recall means explaining and applying basic concepts again without relying on memorized recipes.

338 knowledge transfer (W22)
Knowledge transfer enables another person to understand and apply a concept, while checking their independent reasoning.

Sources and publication policy

This PDF follows the updated complementary-task roadmap in repository revision b28ca07 and PR #15. The 44 source definitions organize concepts; they are not duplicate production assignments for the two people.

Updated roadmap review
https://github.com/Prem-Tomar/fieldops-mastery/pull/15

Structured roadmap
https://github.com/Prem-Tomar/fieldops-mastery/blob/docs/product-driven-roadmap/curriculum/roadmap.json

Concept catalog
https://github.com/Prem-Tomar/fieldops-mastery/blob/docs/product-driven-roadmap/curriculum/catalog.json

JavaScript guide
https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide

TypeScript handbook
https://www.typescriptlang.org/docs/handbook/intro.html

React learning guide
https://react.dev/learn

React Native core components
https://reactnative.dev/docs/components-and-apis

Node event-loop guidance
https://nodejs.org/en/learn/asynchronous-work/dont-block-the-event-loop

PostgreSQL tutorial
https://www.postgresql.org/docs/current/tutorial.html

PostgreSQL transaction isolation
https://www.postgresql.org/docs/current/transaction-iso.html

PostgreSQL query-plan guidance
https://www.postgresql.org/docs/current/using-explain.html

Publication remains paused. Do not create additional issues or activate automatic issue creation until Prem-Tomar authorizes it. Existing successors are reused rather than duplicated.

Scope: the 22-week core roadmap only. An AI/ML extension is outside this document. Future issues add exact function contracts, examples, hints and design screens as appropriate.